

Adaptation of Uncoordinated Distance-Vector Routing for Unencrypted Amateur Radio Networks

Abstract

This paper describes a decentralized, unencrypted mesh networking protocol designed for bandwidth-constrained, half-duplex amateur radio environments (e.g., 1200 baud AFSK over VHF/UHF). Drawing heavily from the routing logic of the Reticulum Network Stack (RNS)—specifically Uncoordinated Distance-Vector Routing via Announce Propagation—this protocol strips away cryptographic dependencies to maintain strict compliance with FCC Part 97 cleartext regulations. The architecture relies on binary data serialization, sequential message identification, application-layer acknowledgments, and decentralized route discovery to create a highly resilient, low-overhead mesh network.

1. Introduction

Traditional IP-based routing is poorly suited for tactical or ad-hoc amateur radio networks using hardware like Baofeng UV-5R transceivers paired with KISS-compatible Terminal Node Controllers (TNCs). Protocols requiring constant "hello" packets and full network map synchronization (e.g., OSPF, OLSR) consume excessive airtime. To resolve this, we utilize an Uncoordinated Distance-Vector Routing approach. Nodes do not maintain synchronized topology maps. Instead, routing tables are an emergent property of periodic "Announce" packets washing over the network. To meet the technical constraints of low-baud radio and the regulatory constraints of amateur radio, this protocol replaces Reticulum's cryptographic hashes with plaintext callsigns and utilizes a highly packed binary structure to minimize on-air transmission time.

2. Core Protocol Mechanics

2.1 Route Discovery (Announce Propagation)

The network operates without central infrastructure. Route discovery is achieved through periodic announcements.

- **The Announce Packet:** A node (Node A) broadcasts an Announce packet containing its callsign, a sequence ID, and a hop count initially set to 0.

- **Propagation:** When an adjacent node (Node B) receives this packet, it records Node A as being 1 hop away. If Node B has not seen this exact sequence ID from Node A recently, it increments the hop count to 1, replaces the "Transmitter" field with its own callsign, and rebroadcasts the packet.
- **Path Selection:** If a node receives multiple Announce packets for the same destination, it strictly prefers the route with the lowest hop count. In the event of a tie, it prefers the "first-heard" route.

2.2 Loop Prevention

To prevent broadcast storms and infinite routing loops, the protocol implements Sequential Message IDs. Every node maintains an internal 16-bit counter (0 to 65535) for its outgoing packets. Every generated packet includes the Originator's Callsign and this Sequence ID. Receiving nodes maintain a short-term cache of [Origin_Callsign + Sequence_ID]. If a node receives a packet (Announce, Broadcast, or Direct) whose signature is already in the cache, the packet is silently discarded.

2.3 Route Expiration (Time-To-Live)

Due to the dynamic nature of mobile radios, paths degrade. Routing table entries are subject to a strict 30-minute Time-To-Live (TTL) (1800 seconds). Each time an Announce is received for a destination, its TTL timer is reset to 30 minutes. If 30 minutes elapse without a fresh Announce for a destination, the route is purged from the node's memory.

2.4 Reliability and Retries

Directly addressed packets operate in a "Connected Mode" utilizing Application-Layer Acknowledgments (ACKs).

- **Fire and Wait:** When Node A sends a Direct message to Node C, it starts a local retry timer (recommended: 5 seconds + random jitter to prevent collisions).
- **Acknowledgment:** Upon successful receipt and decoding of the Direct message, the final destination (Node C) immediately generates an ACK packet addressed back to Node A. Intermediate routing nodes do not generate ACKs; they only forward them.
- **Retries:** If Node A does not receive the ACK before the timer expires, it retransmits the message with the same Sequence ID. This occurs up to a maximum of 3 retries before the transmission is deemed failed.

3. Data Serialization and Frame Structure

To maximize efficiency over a 1200 baud Bell 202 modem, all packets are serialized in a raw Binary Format before being encapsulated in a standard KISS frame (using 0xC0 FEND and 0x00

Data Frame delimiters).

ITA2 (Baudot) Compression: To significantly reduce overhead, Callsigns and Payloads utilize ITA2 encoding rather than standard 8-bit ASCII. Since ITA2 uses 5 bits per character, 8 characters (a standard 6-character callsign plus two LTRS/FIGS shift markers) can be bit-packed perfectly into 5 bytes (40 bits).

3.1 Common Packet Header

All packets begin with a 13-byte standard header.

Byte Offset	Length	Field	Description / Data Type
0	1 byte	Packet Type	0x01=Announce, 0x02=Broadcast, 0x03=Direct, 0x04=ACK
1	5 bytes	Origin Callsign	Bit-packed ITA2, max 8 chars including shifts. Padded with NULL (00000).
6	5 bytes	Transmitter Callsign	The node transmitting this specific hop. Bit-packed ITA2.
11	2 bytes	Sequence ID	Unsigned 16-bit integer (Big-Endian). Loop prevention.

3.2 Packet Type 1: Announce (14 Bytes Total)

Used to build the routing table.

Byte Offset	Length	Field	Description
0 - 12	13 bytes	Common Header	Type = 0x01.
13	1 byte	Hop Count	Unsigned 8-bit integer. Origin sets to 0.

3.3 Packet Type 2: Broadcast (Variable Length)

An unconnected, unacknowledged message to anyone in RF range.

Byte Offset	Length	Field	Description
0 - 12	13 bytes	Common Header	Type = 0x02.
13+	Variable	Payload	Plaintext ITA2 bit-packed message. Approx 37.5% smaller than ASCII.

3.4 Packet Type 3: Direct Message (Variable Length)

A connected, routed message to a specific destination.

Byte Offset	Length	Field	Description
0 - 12	13 bytes	Common Header	Type = 0x03.
13	5 bytes	Dest Callsign	Final recipient. Bit-packed ITA2.
18	5 bytes	Next Hop Callsign	The neighbor node receiving the handoff. Bit-packed ITA2.
23+	Variable	Payload	Plaintext ITA2 bit-packed message.

3.5 Packet Type 4: Acknowledgment (ACK) (20 Bytes Total)

Sent by the final destination back to the originator.

Byte Offset	Length	Field	Description
0 - 12	13 bytes	Common Header	Type = 0x04.
13	5 bytes	Dest Callsign	The Origin node of the original message. Bit-packed ITA2.
18	2 bytes	Acked Seq ID	The Sequence ID of the message being confirmed.

4. State Machine & Node Logic

An AI implementing this protocol should adhere to the following event-driven state machine.

Event: Receive Announce (Packet Type 0x01)

1. Extract Origin, Sequence ID, Transmitter, and Hop Count. Decode callsigns from ITA2 format.
2. Check Origin against the local Callsign. If it matches, drop (do not route self).
3. Check [Origin + Sequence ID] against the Recent Packets Cache. If present, drop (loop prevention).
4. Add [Origin + Sequence ID] to the cache.
5. Check Routing Table for Origin. If Origin does not exist, OR if Hop Count < Existing Hop Count:
 - Update Routing Table: Destination = Origin, Next_Hop = Transmitter, Hops = Hop Count, Last_Seen = Current_Time.
 - Modify packet: Increment Hop Count by 1. Update Transmitter to local Callsign (re-encode to ITA2).
 - Queue packet for TNC transmission.
6. If Hop Count >= Existing Hop Count, do nothing (drop packet).

Event: Receive Direct Message (Packet Type 0x03)

1. Extract Destination and Next_Hop (Decode from ITA2).
2. Check [Origin + Sequence ID] against cache. If present, drop. Add to cache.
3. If Destination == Local_Callsign:
 - Message has reached its target. Decode payload from ITA2 and pass to the user application.
 - Generate an ACK (Type 0x04) targeting Origin. Look up Origin in Routing Table to find the Next_Hop for the return path. Queue ACK for transmission.
4. If Next_Hop == Local_Callsign (Node is a router in this path):
 - Look up Destination in Routing Table.
 - If route exists: Update Next_Hop in the packet header to the value from the routing table. Update Transmitter to Local Callsign. Queue for transmission.
 - If no route exists: Drop packet.

Event: Periodic Cleanup (Every 60 Seconds)

1. Iterate through Routing Table. If Current_Time - Last_Seen > 1800 seconds, remove the entry.

- 2. Iterate through Recent Packets Cache. Remove entries older than 5 minutes to prevent memory overflow.

Appendix A: Bit-Level Packet Examples

This section demonstrates the precise, bit-by-bit construction of over-the-air packets.

Reference: ITA2 5-Bit Character Mapping

To follow the bitwise math below, note the standard MSB-first 5-bit ITA2 binary maps used in this implementation:

- **Control/Shifts:** LTRS = 11111, FIGS = 11011, NULL = 00000
- **Letters:** A = 11000, B = 10011, C = 01110, D = 10010, E = 10000, F = 10110, H = 00101, I = 01100, K = 11110, N = 00110, S = 10100, T = 00001, W = 11001, X = 10111, Y = 10101, Z = 10001
- **Numbers (Require FIGS preceding):** 1 = 11101, 2 = 11001, 3 = 10000

Example 1: Broadcast Packet (Unconnected)

Scenario: Node **K1ABC** sends a broadcast message with sequence ID 0x0001 and payload "HI".

- **Callsign Compilation (K1ABC):** 8 characters total: LTRS + K + FIGS + 1 + LTRS + A + B + C Binary: 11111 11110 11011 11101 11111 11000 10011 01110 Total bits: 40 (Exactly 5 bytes).
- **Payload Compilation (HI):** 3 characters total: LTRS + H + I Binary: 11111 00101 01100. (15 bits). Needs 1 bit of 0 padding to align to the nearest byte (16 bits / 2 bytes). Padded Binary: 11111001 01011000

Complete Bit-by-Bit Frame (15 Bytes Total):

Byte #	Hex	Binary	Field	Bit-Level Breakdown & Packing Math
00	02	00000010	Packet Type	0x02 (Broadcast)
01	FF	11111111	Origin (K1ABC)	All 5 bits of LTRS (11111) + first 3 bits of K (111)
02	B7	10110111	Origin (cont)	Last 2 bits of K (10) + all 5 of FIGS (11011) + 1st bit of 1 (1)

Byte #	Hex	Binary	Field	Bit-Level Breakdown & Packing Math
03	BF	10111111	Origin (cont)	Last 4 bits of 1 (1101) + first 4 bits of LTRS (1111)
04	E2	11100010	Origin (cont)	Last bit of LTRS (1) + all 5 of A (11000) + first 2 of B (10)
05	6E	01101110	Origin (End)	Last 3 bits of B (011) + all 5 of C (01110)
06	FF	11111111	Transmitter	<i>(Exact same 5-byte packing as Origin since K1ABC is the transmitter)</i>
07	B7	10110111	Transmitter	
08	BF	10111111	Transmitter	
09	E2	11100010	Transmitter	
10	6E	01101110	Transmitter	
11	00	00000000	Seq ID High	Sequence 0x0001 (High Byte)
12	01	00000001	Seq ID Low	Sequence 0x0001 (Low Byte)
13	F9	11111001	Payload	All 5 bits of LTRS (11111) + first 3 bits of H (001)
14	58	01011000	Payload	Last 2 of H (01) + all 5 of I (01100) + 1 padding bit (0)

Example 2: Direct Message (Connected)

Scenario: Node **K1ABC** sends a direct message to **N3DEF** with sequence ID 0x0005. The routing table indicates the next hop is **W2XYZ**. The payload is "**TEST**".

- **Dest Callsign (N3DEF):** LTRS N FIGS 3 LTRS D E F Binary: 11111 00110 11011 10000 11111 10010 10000 10110
- **Next Hop Callsign (W2XYZ):** LTRS W FIGS 2 LTRS X Y Z Binary: 11111 11001 11011 11001 11111 10111 10101 10001
- **Payload (TEST):** LTRS T E S T Binary: 11111 00001 10000 10100 00001 (25 bits). Needs 7 padding 0s to reach 32 bits (4 bytes). Padded Binary: 11111000 01100001 01000000 10000000

Complete Bit-by-Bit Frame (27 Bytes Total):

Byte #	Hex	Binary	Field	Bit-Level Breakdown & Packing Math
00	03	00000011	Packet Type	0x03 (Direct Message)
01-05	...	(See Ex 1)	Origin	K1ABC encoded to FF B7 BF E2 6E
06-10	...	(See Ex 1)	Transmitter	K1ABC encoded to FF B7 BF E2 6E (since it's the 1st hop)
11	00	00000000	Seq ID High	0x0005
12	05	00000101	Seq ID Low	0x0005
13	F9	11111001	Destination	LTRS (11111) + first 3 of N (001)
14	B7	10110111	Destination	Last 2 of N (10) + FIGS (11011) + first 1 of 3 (1)
15	0F	00001111	Destination	Last 4 of 3 (0000) + first 4 of LTRS (1111)
16	CA	11001010	Destination	Last 1 of LTRS (1) + D (10010) + first 2 of E (10)
17	16	00010110	Destination	Last 3 of E (000) + F (10110)
18	FE	11111110	Next Hop	LTRS (11111) + first 3 of W (110)
19	77	01110111	Next Hop	Last 2 of W (01) + FIGS (11011) + first 1 of 2 (1)
20	9F	10011111	Next Hop	Last 4 of 2 (1001) + first 4 of LTRS (1111)
21	DE	11011110	Next Hop	Last 1 of LTRS (1) + X (10111) + first 2 of Y (10)
22	B1	10110001	Next Hop	Last 3 of Y (101) + Z (10001)
23	F8	11111000	Payload	LTRS (11111) + first 3 of T (000)
24	61	01100001	Payload	Last 2 of T (01) + E (10000) + first 1 of S (1)

Byte #	Hex	Binary	Field	Bit-Level Breakdown & Packing Math
25	40	01000000	Payload	Last 4 of S (0100) + first 4 of T (0000)
26	80	10000000	Payload	Last 1 of T (1) + 7 bits of zero padding (0000000)

Note: When node W2XYZ receives this packet, it will drop the packet to its TNC, overwrite the Transmitter field (Bytes 06-10) with its own ITA2-encoded callsign, overwrite the Next Hop field (Bytes 18-22) with the next node in the distance-vector table toward N3DEF, and then re-transmit.

Appendix B: Announce Packet Propagation Trace (4-Node Network)

This section traces a single Announce packet originating from Node A as it propagates through a 4-node network (Nodes A, B, C, and D). It specifically illustrates how the protocol resolves receiving the same announcement through multiple paths to prevent routing loops and identify the most efficient route.

Network Topology Assumption: Node A is in direct RF range of Nodes B and C. Node D is in direct RF range of Nodes B and C (but cannot hear Node A directly).

- **Step 1: Node A Initiates Announce** Node A wishes to announce its route to the mesh. It generates a new Sequence ID (e.g., 0x0001). Packet Header: `Origin = A`, `Transmitter = A`, `Seq = 0x0001`, `Hops = 0`. Node A transmits the packet over the air.
- **Step 2: Nodes B and C Receive** Both Node B and Node C hear Node A's transmission. They check their Recent Packets Cache for the signature `[A + 0001]`. Neither has seen it. They log `[A + 0001]` in their respective caches. They update their routing tables: Destination A is reachable via Next Hop A at 1 hop. They prepare the packet for rebroadcast: Hop Count becomes 1, Transmitter becomes B (for Node B's packet) and C (for Node C's packet).
- **Step 3: Nodes B and C Rebroadcast** Node B transmits its packet: `Origin = A`, `Transmitter = B`, `Seq = 0x0001`, `Hops = 1`. Node C transmits its packet: `Origin = A`, `Transmitter = C`, `Seq = 0x0001`, `Hops = 1`.
- **Step 4: Node D Hears Multiple Paths (Collision Resolution)** Because Node D is in range of both B and C, it will hear both rebroadcasts. RF propagation delays and hardware timing mean one will arrive slightly before the other.
 - **First Arrival (e.g., from Node B):** Node D receives the packet from Transmitter B. Node D evaluates the cache for `[A + 0001]`. It is not found. Node D logs `[A +`

0001] in its cache. Node D updates its routing table: Destination A is mapped to Next Hop B at 2 hops (Hops 1 + 1). Node D queues the packet for its own rebroadcast (Origin = A, Transmitter = D, Hops = 2).

- **Second Arrival (e.g., from Node C):** Milliseconds later, Node D receives the packet from Transmitter C. Node D evaluates the cache for [A + 0001]. Resolution: Because [A + 0001] was just cached during Node B's transmission, the loop prevention mechanism immediately triggers. Node D silently discards Node C's packet. It does not update its routing table to Node C, nor does it rebroadcast Node C's packet.

Result: This emergent behavior naturally enforces the protocol's "first-heard" rule. Node D locks onto Node B's path because it was the fastest and first clear route established for that sequence ID, avoiding broadcast storms while dynamically building an effective Distance-Vector map back to Node A.